Basic Introduction
Accessing Documents
General Information
STREAM
DELTA PRO
DELTA 2
DELTA 2 MAX
Smart Home Panel
Smart Plug
PowerStream Micro-inverter
Wave Air Conditioner
Glacier
Power Kits
PowerOcean
DELTA Pro Ultra
DELTA Pro 3
Smart Home Panel II
River 2 Pro
Code package download
Support

## Host

| Key | Value |
| --- | --- |
| host | https://api-e.ecoflow.com |

## HTTP access steps

Create a signature with accesskey and secretkey for authentication when HTTP requesting.

How to implement signature:

**Step 1**: request parameters must be sorted by ASCII value and concatenated with characters =, &

E.g. str=param1=value1&param2=value2

**Step 2**: if the type is nested, expand and splice according to the method of step 1.

E.g. deviceInfo.id=1&deviceList[0].id=1&deviceList[1].id=2&ids[0]=1&ids[1]=2&ids[2]=3&name=demo1

```
{
    // General Parameter, expand to: name=demo1
    "name" : "demo1" ,
    // Array, expand to: ids[0]=1&ids[1]=2&ids[2]=3
    "ids" :[1, 2, 3],
    // Object, expand to: deviceInfo.id=1
    "deviceInfo" :{
        "id" :1
    },
    // Object Array, expand to: deviceList[0].id=1&deviceList[1].id=2
    "deviceList" :[
        {
            "id" :1
        },
        {
            "id" :2
        }
    ]
}
```

**Step 3**: concatenate accessKey, nonce, timestamp

E.g. str=param1=value1&param2=value2&accessKey=***&nonce=...&timestamp=...

**Step 4**: encrypt

E.g. byte[] signBytes = HMAC-SHA256(str, secretKey)

**Step 5**: convert byte[] to hexadecimal string. String sign = bytesToHexString(signBytes)

E.g. sign=85776ede686fe4783eac48135b0b1748ba2d7e9bb7791b826dc942fc29d4ada8

**Step 6**: add accessKey, nonce, timestamp, sign to HTTP header

E.g.

```
// HTTP Header
accessKey:ba2d7e9...
nonce:123456
timestamp:1671171709428
sign:85776ede686fe...
```

**Step 7**: get request data according to Content-Type header

If Content-Type is "application/json;charset=UTF-8", get request data from request body, else get from request query string.

E.g.

```
// HTTP header content-type=application/json;charset=UTF-8
// HTTP body
{
 "name": "...",
 "begin_time": 123456789
}

// HTTP header content-type is none or other
// HTTP query string
https://${host}//iot-open/sign/device/quota/all?sn=123
```

**Step8**: verify that the steps to generate the signature are correct

```
accessKey=Fp4SvIprYSDPXtYJidEtUAd1o
secretKey=WIbFEKre0s6sLnh4ei7SPUeYnptHG6V
nonce=345164
timestamp=1671171709428

// HTTP json Body
{
    "sn" : "123456789" ,
    "params" :{
        "cmdSet" :11,
        "id" :24,
        "eps" :0
    }
}

// key-value string
params.cmdSet=11&params.eps=0&params.id=24&sn=123456789&accessKey=Fp4SvIprYSDPXtYJidEtUAd1o&nonce=345164&timestamp=1671171709428
```

```
// sign. If the signature you generated is this value, than is correct
07c13b65e037faf3b153d51613638fa80003c4c38d2407379a7f52851af1473e
```

General HTTP Header

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| accessKey | string | Yes | Get from IoT platform |
| nonce | int | Yes | Random number, 6 digits. Such as 537642 |
| timestamp | long | Yes | UTC timestamp. Such as 1672382607378 |
| sign | string | Yes | Creating by the signature rule |

Query the user's bound device list

Only returns the device bound to itself, not by share.

| Key | Value |
| --- | --- |
| Method | GET |
| URL | https://${host}/iot-open/sign/device/list |
| Content-Type | none |

Request Body

None

Response Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| code | string | Yes | 0: success |
| message | string | | |
| data | List | | Device's info list |

DeviceInfo

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| sn | string | Yes | Device's sequence number |
| deviceName | string | Yes | Device's name |
| online | int | Yes | Device online or not0: No, 1: Yes |

Example

```
// request
curl -X GET https://api-e.ecoflow.com/iot-open/sign/device/list \
-H 'accessKey:OCHzRuj6NLF7o43***' \
-H 'timestamp:1681796503289' \
-H 'nonce:234762' \
-H 'sign:f560c3e31d96ad31e4567939f9b3dca7b2c454ca7003f60***'

// response
{
    "code":"0",
    "message":"Success",
    "data":[
        {
            "sn":"DCABZ****",
            "online":1
        }
    ]
}
```

Setting device's function

| Key | Value |
| --- | --- |
| Method | PUT |
| URL | https://${host}/iot-open/sign/device/quota |
| Content-Type | application/json;charset=UTF-8 |

Request Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| sn | string | Yes | Device's sequence number |
| params | SetCmdRequest | Yes | |

Response Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| code | string | Yes | 0: success |
| message | string | | |

Example

```
// Delta Pro: Setting the X-Boost switch
// Request
curl -X PUT https://api-e.ecoflow.com/iot-open/sign/device/quota \
-H 'Content-Type:application/json;charset=UTF-8' \
-H 'accessKey:OCHzRuj6NLF7o43***' \
-H 'timestamp:1681872798000' \
-H 'nonce:238752' \
-H 'sign:0f3a1b102cd9a306307b7a9a0f0a9add7b8bfc93cf11***' \
-d '{"sn":"DCABZ***","params":{"cmdSet":32,"id":66,"enabled":1}}'

// Response
{
    "code":"0",
    "message":"Success"
}
```

### Query the device's quota infomation

| Key | Value |
| --- | --- |
| Method | POST |
| URL | https://${host}/iot-open/sign/device/quota |
| Content-Type | application/json;charset=UTF-8 |

Request Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| sn | string | Yes | Device's sequence number |
| params | GetCmdRequest | Yes | |

Response Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| code | string | Yes | 0: success |
| message | string | | |
| data | GetCmdResponse | | |

Example

```
// Delta Pro: Get the X-Boost switch value
// Request
curl -X POST https://api-e.ecoflow.com/iot-open/sign/device/quota \
-H 'Content-Type:application/json;charset=UTF-8' \
-H 'accessKey:OCHzRuj6NLF7o43***' \
-H 'timestamp:1681872798000' \
-H 'nonce:238752' \
-H 'sign:1e48307441d06a6b31d4c0d66be1580c7748fd0***' \
-d '{"sn":"DCABZ***","params":{"cmdSet":32,"id":66,"quotas":["inv.cfgAcEnabled"]}}'

// Response
{
    "code":"0",
    "message":"Success",
    "data":{
        "inv.cfgAcEnabled":1
    }
}
```

### Query device's all quota infomation

| Key | Value |
| --- | --- |
| Method | GET |
| URL | https://${host}/iot-open/sign/device/quota/all |
| Content-Type | none |

Request Query String

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| sn | string | Yes | Device's sequence number |

Response Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| code | string | Yes | 0: success |
| message | string | | |
| data | GetAllQuotaResponse | | |

Example

```
// Get Delta Pro all quota values
// Request
curl -X GET https://api-e.ecoflow.com/iot-open/sign/device/quota/all?sn=DCABZ*** \
-H 'accessKey:OCHzRuj6NLF7o43***' \
-H 'timestamp:1681872798000' \
-H 'nonce:238752' \
-H 'sign:534215ccfe93979588975630ea0ad7091c7e250ae5***'

// Response
{
    "code":"0",
    "message":"Success",
    "data":{
        "bmsMaster.soc":"100",
        "bmsMaster.temp":"34",
        "bmsMaster.inputWatts":"0",
        "bmsMaster.outputWatts":"0",
        "pd.remainTime":"14781",
        "inv.cfgAcEnabled":"0",
        "mppt.carState":"0",
        "pd.usb1Watts":"0",
        "pd.usb2Watts":"0",
        "pd.qcUsb1Watts":"0",
        ...
    }
}
```

### MQTT certificate acquisition

Get the MQTT certification, using it for MQTT communication.

| Key | Value |
| --- | --- |
| Method | GET |
| URL | https://${host}/iot-open/sign/certification |

| Key | Value |
| --- | --- |
| Content-Type | none |

Request Body

None

Response Body

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| code | string | Yes | 0: success |
| message | string | | |
| data | AppCertification | | |

AppCertification

| Field | Field's Type | Required | Description |
| --- | --- | --- | --- |
| certificateAccount | string | Yes | MQTT certification account |
| certificatePassword | string | Yes | MQTT certification password |
| url | string | Yes | MQTT host |
| port | string | Yes | MQTT port |
| protocol | string | Yes | MQTT communication protocol: mqtt、mqtts |

Example

```
// Request
curl -X GET https://api-e.ecoflow.com/iot-open/sign/certification \
-H 'accessKey:OCHzRuj6NLF7o43***' \
-H 'timestamp:1681872798000' \
-H 'nonce:238752' \
-H 'sign:0396ef9748bf273ecf6ad7ded6333f343d778945f7***'

// Response
{
    "code":"0",
    "message":"Success",
    "data":{
        "certificateAccount":"open-57c134518b5***",
        "certificatePassword":"959253cc103a4008***",
        "url":"mqtt.ecoflow.com",
        "port":"8883",
        "protocol":"mqtts"
    }
}
```

MQTT Topic

| Usage of Topic | Topic | From | To |
| --- | --- | --- | --- |
| Report device quota | /open/${certificateAccount}/${sn}/quota | device | app |
| Set device function | /open/${certificateAccount}/${sn}/set | app | device |
| Reply to the set result | /open/${certificateAccount}/${sn}/set_reply | device | app |
| Get some quota | /open/${certificateAccount}/${sn/get | app | device |
| Reply to the get result | /open/${certificateAccount}/${sn}/get_reply | device | app |
| Report Device status | /open/${certificateAccount}/${sn}/status | device | app |

Description

| Field | Get from the HTTP** **API |
| --- | --- |
| certificateAccount | GET: /iot-open/sign/certification |
| sn | GET: /iot-open/sign/device/list |

Request Topic's Common Field

| Field | Field Type | Description |
| --- | --- | --- |
| id | string | Message ID, unique per request.Such as UTC timestamp: 1672389360845Response will return the same id |
| version | string | Protocol version. Input 1.0 at this moment |
| operateType | string | Device's operator typeDelta Pro, Smart Home Panel device input TCP |
| from | string | Request from source. Such as iOS, Android, Web |
| params | ParamInfo | |

Example

```
{
  "id":"123456789",
  "version":"1.0",
  "operateType":"TCP",
  "from":"iOS",
  "params": {
    "cmdSet": 32,
    "id": 33,
    "standByMode": 0
  }
}
```

3.Reply Topic's Common Field

| Field | Field Type | Description |
| --- | --- | --- |
| id | string | Message ID, unique per request.Such as UTC timestamp: 1672389360845Response will return the same id |
| version | string | Protocol version. Input 1.0 at this moment |
| operateType | string | Device's operator typeDelta Pro, Smart Home Panel device input TCP |
| code | string | Response result0: success-1: this device don't belong to user-2: device offline |
| message | string | Error information |
| data | DataInfo | Response data |

Example

```
{
    "id":123456789,
    "version":"1.0",
    "operateType": "TCP",
    "code": "0",
    "data": {
        "cmdSet": 32,
        "id": 33,
        "ack": 0
    }
}
```

Report Device status

| Usage of Topic | Topic | From | To |
| --- | --- | --- | --- |
| Report Device status | /open/${certificateAccount}/${sn}/status | device | app |

Message Format

```
{
  "id":"123456789",
  "version":"1.0",
  "timestamp": 1634841971000,
  "params": {
    "status": 0
  }
}
```

Field

| Field | Field Type | Description |
| --- | --- | --- |
| status | int | Device online or not0: No, 1: Yes |

## Was this page helpful?

YES NO

## Was this page helpful?

We'd like to hear from you.Click here to give your feedback or suggestions.